SIET CSE DEPARTMENT PLATFORM

NATTU — IMPLEMENTATION GUIDE

Student Portal • Database Import • Storage • OD • Activities • Reviews • QR Check-in

Goal: turn the existing student UI into a real database-backed student workflow.

Based on PRD v2.0 and API Contract v2.0.

1. How to use this guide

Do one task at a time. Copy the prompt into your coding assistant, inspect the plan, implement, test, and only then move on.

Do not invent backend behavior. Meena owns the HOD decision side; you consume the shared API contract.

Stop and ask whenever a schema field, endpoint, status, permission, upload rule, or workflow is unclear.

2. Your exact ownership

You own	You coordinate with Meena on
Supabase client + shared types	Schema/API changes
Real student import	Auth/profile behavior
Storage buckets + document UI	HOD document preview
Apply OD + My ODs + resubmit	OD decision response shapes
Student activity forms/details	Activity lifecycle
Student reviews + progress	Review sessions
QR check-in	HOD QR generation
Student notifications UI	End-to-end testing

3. Questions to settle before coding

Questions to ask before coding

■ What exact real-student import file/format is the final source of truth?

■ What is the final upload limit and allowed MIME types?

■ Should OD use exact times only, or slot types such as Full Day / Forenoon / Afternoon / Periods?

■ Are project/hackathon/internship teammates selected only from real student records?

■ Are weekly reviews strictly non-evaluative with no scores?

4. Phase 0 — Shared setup

Prompt 0A — inspect student-side code

```
Inspect the existing SIET CSE repository before changing anything.
Focus on:
1. /student routes,
2. DataContext/localStorage usage,
3. mock student identity,
4. OD form,
5. My OD list/detail,
6. activity pages,
7. review pages,
8. document upload code,
9. existing TypeScript types.
Do not modify code yet.
Return a simple migration checklist and identify any hard-coded student IDs or fake document
URLs.
```

Prompt 0B — shared types

```
Using the PRD v2.0 and API Contract v2.0, inspect the current TypeScript types.
Create/reconcile shared types for User, Student, ODRequest, Activity, ReviewSession,
WeeklyProgress, Document, Notification.
Do not invent fields that are not supported by the current contract/schema.
List any type conflicts that require Meena's confirmation before changing.
```

5. Phase 0 — Import real students

Prompt 0C — student import script

```
Create a safe import script for the department's real student data.
Target structure:
- Year I: A-E
- Year II: A-E
- Year III: A-C
- Year IV: A-B
Requirements:
- Validate register number uniqueness.
- Validate year and section values.
- Do not create duplicate students.
- Produce a dry-run summary before inserting.
- Keep the import repeatable/idempotent.
- Do not hard-code students into React components.
```

Done when

- ■ Dry run shows the expected 15 sections.
- ■ Duplicate register numbers are detected.
- ■ Import can be repeated without duplicating students.
- ■ Student records are available through Supabase after import.

6. Phase 1 — Student authentication/profile

Prompt 1 — authenticated student

```
Replace the mock student session with Supabase Auth integration.
Use GET /api/v1/me to obtain the authenticated profile.
Requirements:
- Never send student_id from a form as the source of identity.
- Never allow the client to choose the role.
- Student pages must show the authenticated student's real profile.
- Preserve the existing UI as much as possible.
First show the files you will change.
```

7. Phase 1 — Apply OD

Prompt 2 — real Apply OD

```
Connect the existing Apply OD form to POST /api/v1/od-requests.
Use the API Contract v2.0 as the exact request contract.
Requirements:
- Server derives the authenticated student.
- New submission becomes PENDING.
- Send event/activity/team/document references only when valid.
- Show API validation errors clearly.
- After success, navigate to the OD detail/status page.
Do not create a fake OD ID on the client.
```

Done when

- ■ A real student can submit an OD.
- ■ The database contains the submission.
- ■ The OD belongs to the authenticated student.

- Refreshing the page does not lose the request.
- The HOD can see the request.

Prompt 3 — My ODs + timeline

```
Implement My OD Requests using GET /api/v1/od-requests and GET /api/v1/od-requests/{id}.
Display:
- business code,
- purpose/event,
- status,
- remarks/rejection reason/revision notes,
- documents,
- status timeline.
Use the real API response. Do not reconstruct history from frontend state.
```

Prompt 4 — resubmit

```
Implement the resubmit path.
Show Edit & Resubmit only when the current status is REVISION_REQUESTED.
Use PUT /api/v1/od-requests/{id}/resubmit.
After success, the status should be PENDING.
Do not allow resubmission from APPROVED or REJECTED unless the API contract explicitly changes.
```

8. Phase 2 — Documents

Prompt 5 — secure document upload

```
Implement document upload using Supabase Storage and the documents API.
Requirements:
- Store actual files in private storage.
- Store metadata in documents.
- Upload only to an authorized owner record.
- Use GET /api/v1/documents/{id}/url for temporary signed access.
- Never expose a permanent public storage URL.
- Show upload progress and useful validation errors.
Use the final agreed file size and MIME-type rules; if they are not finalized, stop and ask me.
```

9. Phase 3 — Activities

Prompt 6 — student activities

```
Implement student-side Projects, Hackathons, and Internships.
Use:
POST /api/v1/activities
GET /api/v1/activities
GET /api/v1/activities/{id}
Requirements:
- Keep activity types exactly aligned with the contract.
- Team members must reference real student records.
- Support technologies, relevant organization/company fields, GitHub/demo links, dates,
guide/mentor fields supported by the schema.
- Show the activity status timeline returned by the backend.
Do not create a second activity lifecycle in the frontend.
```

10. Phase 4 — Weekly Reviews

Prompt 7 — weekly progress

```
Implement the student weekly progress form for a scheduled review session.
Use POST /api/v1/reviews/sessions/{id}/progress.
Required fields:
- completed_this_week
- currently_working_on
- next_week_goal
- blockers
```

```
Optional:
- github_url
Show previous submitted progress when available.
Keep the review non-evaluative; do not add scores unless the team has explicitly resolved that
open decision.
```

Prompt 8 — QR check-in

```
Implement student QR check-in.
The HOD generates the dynamic token; the student scans it.
Use POST /api/v1/reviews/sessions/{id}/check-in.
Requirements:
- Never derive or guess the token from session ID/date.
- Send the scanned token to the API.
- Show success/failure clearly.
- Prevent duplicate local submissions while still trusting the server as the source of truth.
```

11. Phase 1–5 — Notifications

Prompt 9 — notifications

```
Implement the student notification UI using:
GET /api/v1/notifications
PATCH /api/v1/notifications/{id}/read
Show OD decisions, submission updates, review reminders, and progress deadlines returned by the
API.
Do not hard-code the HOD's name or a student recipient.
```

12. Integration with Meena

Before each integration, send Meena the exact request/response assumptions you are using. If the backend differs, align the contract first rather than patching around it in the UI.

- Auth: confirm the /me response and session behavior.
- OD: confirm POST, GET, resubmit, and decision response shapes.
- Documents: confirm owner_id, document_type, and signed URL behavior.
- Activities: confirm team member representation and lifecycle fields.
- Reviews: confirm session IDs, QR token response, and progress/check-in responses.

13. Final Nattu end-to-end test

Done when

- Real student can log in and sees their own profile.
- Student submits an OD and it survives refresh.
- Student sees HOD approval/rejection/revision notes.
- Revision-requested OD can be edited and resubmitted.
- Documents upload privately and can be previewed via signed URL.
- Activity team members map to real student records.
- Student can submit weekly progress.
- Student can check in with a valid dynamic QR token.
- Student cannot read another student's private records through direct API calls.

14. If you get stuck: use this prompt

Universal debugging prompt

```
I am implementing the student side of the SIET CSE Department Platform.
Task: [TASK]
Error/result: [ERROR]
Relevant code: [PASTE]
Use the PRD v2.0 and API Contract v2.0 as the source of truth.
First explain the issue simply.
Then give the smallest safe fix.
Do not change the database schema, API contract, permissions, or status machine without
identifying the change and asking me to confirm.
Give me 3 tests after the fix.
```